AI POWER NEWS ASSISTANT
REPORT

A Python-Based Topic-Driven News Retrieval System

Technologies: Python • NewsData.io REST API • JSON • urllib • urllib.parse

Prepared by

Abhishek Singh

B.Tech – Computer Science / Artificial Intelligence

Professional Academic Project Documentation

Project Scope

Topic-based news retrieval and display of the first ten articles. Voice Search, Text-to-Speech, and Audio News are not included in the current implementation.

CERTIFICATE

Certificate

This is to certify that the project entitled **"AI Power News Assistant"** is a Python-based software project developed for topic-driven news retrieval using the NewsData.io REST API. The project demonstrates practical implementation of Python programming, web API integration, JSON processing, URL parameter encoding, input validation, and structured console output.

Project scope

The documented system accepts a news topic from the user, validates the input, communicates with the configured NewsData.io endpoint, processes the returned JSON response, and displays up to ten news articles. Voice search, Text-to-Speech, and audio news are explicitly outside the current implementation.

DECLARATION

Declaration

I hereby declare that this report presents the design and implementation of the **AI Power News Assistant**, a Python application created for educational and professional project documentation.

Implementation statement

The current implementation is limited to text-based topic search and news retrieval. It does not contain voice-based search, speech recognition, Text-to-Speech, audio news, AI-generated summaries, or personalized recommendation functionality.

Professional responsibility

The project should be used in accordance with the API provider's terms, data policies, authentication requirements, and usage limits. API credentials must be protected and must not be published in a public repository.

ACKNOWLEDGEMENT

Acknowledgement

I express my sincere gratitude to my faculty members, mentors, classmates, family, and all individuals who provided guidance and encouragement during the development and documentation of this project.

Learning experience

The project provided practical exposure to Python programming, HTTP communication, REST APIs, JSON data, URL encoding, input validation, software testing, security practices, and GitHub documentation.

Technical learning

Working with an external API helped connect programming concepts with a practical information-retrieval problem and provided a foundation for future development.

ABSTRACT

Overview

AI Power News Assistant is a Python-based command-line application designed to retrieve topic-specific news through the NewsData.io API. The user enters a topic such as technology, business, sports, science, or artificial intelligence.

Processing

The program validates the topic, creates API parameters containing the API key, query, and language, encodes those parameters using urllib.parse.urlencode(), and sends the request through urllib.request.urlopen().

Data handling

The response is decoded and converted from JSON text into Python data using json.loads(). The program accesses the results collection and selects the first ten available articles. For every article, the title and description are displayed.

Contribution

The project demonstrates a complete basic REST API workflow from user input to structured output. It is intentionally lightweight and focuses on retrieval rather than speech, summarization, verification, or recommendation.

TABLE OF CONTENTS

Report organization

The report is organized into fifteen technical chapters followed by references and an appendix.

Chapter 1

Introduction, motivation, problem statement, objectives, scope, advantages, and report organization.

Chapter 2

Literature and technology review covering news retrieval, REST APIs, JSON, Python, urllib, and NewsData.io.

Chapter 3

Problem analysis covering the existing system, proposed system, assumptions, and constraints.

Chapter 4

Requirements analysis covering hardware, software, functional, and non-functional requirements.

Chapter 5

System design, architecture, workflow, data flow, modules, input, and output design.

Chapter 6

REST API and NewsData.io integration.

Chapter 7

Python implementation and line-by-line explanation of the major program components.

Chapter 8

JSON data processing and article-field extraction.

Chapter 9

Input validation and error-handling strategy.

Chapter 10

Security and API-key management.

Chapter 11

Testing strategy and detailed test cases.

Chapter 12

Results and discussion.

Chapter 13

GitHub documentation and repository structure.

LIST OF FIGURES AND TABLES

Figures

Tables

Note

The diagrams in the report are represented as text-based architecture and workflow models so that the design remains consistent with the actual implementation.

CHAPTER 1 — INTRODUCTION

1.1 Introduction

News is one of the most frequently accessed categories of information on the internet. Users continuously search for updates about technology, business, sports, science, education, politics, entertainment, and other subjects. A program that can retrieve news automatically can reduce repetitive manual searching.

1.2 Need for automated retrieval

Manual browsing requires a user to open a website, enter a search term, inspect results, and repeat the process for different topics. An API-based application can perform the retrieval programmatically and return structured information to another application.

1.3 Project introduction

AI Power News Assistant is designed as a simple topic-based retrieval tool. It uses Python as the application layer and NewsData.io as the external news-data provider.

1.4 Motivation

The project was motivated by the need to understand how a Python program can interact with a real-world REST API. It combines several foundational skills into one practical application.

1.5 Problem statement

Develop a lightweight Python application that accepts a user-selected news topic, validates the input, sends a correctly encoded API request, processes the returned JSON response, and displays up to ten news articles.

1.6 Objectives

The primary objectives are to automate topic-based retrieval, practice API integration, process JSON data, validate input, protect credentials, and produce professional documentation.

1.7 Scope

The current scope is intentionally limited to text-based news retrieval and display. Voice search, Text-to-Speech, audio news, AI summarization, and recommendation systems are not current features.

1.8 Advantages

The system is lightweight, easy to execute, requires no local database, uses standard Python libraries, and demonstrates a clear end-to-end API workflow.

1.9 Report organization

The remaining chapters document technologies, analysis, requirements, design, implementation, testing, security, GitHub documentation, limitations, future scope, and conclusion.

CHAPTER 2 — LITERATURE AND TECHNOLOGY REVIEW

2.1 Online news retrieval

Modern news platforms collect information from many publishers and expose it through websites or APIs.

API-based access is useful when software needs structured data rather than a human-oriented webpage.

2.2 News APIs

A news API provides machine-readable access to articles through HTTP requests. A client usually supplies authentication and search parameters and receives a structured response.

2.3 REST APIs

REST is an architectural style commonly used for web services. Clients communicate with server resources using HTTP methods and receive representations of requested data.

2.4 JSON

JSON is a lightweight text-based data format. Its object and array structures map naturally to Python dictionaries and lists, making it convenient for API responses.

2.5 Python

Python was selected because its syntax is readable and it provides standard libraries for HTTP communication, URL processing, JSON handling, file operations, testing, and automation.

2.6 urllib

The urllib package provides modules for working with URLs and HTTP requests. The project uses urllib.request for the network request.

2.7 urllib.parse

urllib.parse provides URL parsing and encoding utilities. urlencode() converts a dictionary of parameters into a properly encoded query string.

2.8 NewsData.io

NewsData.io is used as the external news service in the project. The provider’s current API documentation should be consulted for endpoint details, authentication, quotas, response fields, and service changes.

2.9 Technology selection

The selected stack is small and appropriate for a foundational API project: Python for logic, urllib for HTTP and encoding, json for parsing, and NewsData.io for news retrieval.

CHAPTER 3 — PROBLEM ANALYSIS

3.1 Existing system

The conventional approach is manual news browsing through search engines, news websites, or applications.

Users perform repeated searches and manually inspect results.

3.2 Problems

Manual searching can be repetitive and does not provide a simple programmable interface for retrieving a fixed number of topic-specific articles.

3.3 Proposed system

The proposed system reduces the workflow to entering a topic and receiving structured output. The program handles request construction and JSON extraction automatically.

3.4 Existing versus proposed

Existing systems prioritize human browsing; the proposed system prioritizes a simple programmatic retrieval workflow.

3.5 Assumptions

The system assumes internet connectivity, a valid API credential, an available API endpoint, and a response containing the expected results structure.

3.6 Constraints

The application depends on an external service, provider quotas, network availability, and the fields returned by the API.

3.7 Project boundary

The system retrieves and presents data. It does not independently determine whether an article is true, generate an AI summary, or provide spoken output.

CHAPTER 4 — REQUIREMENTS ANALYSIS

4.1 Hardware requirements

A normal laptop or desktop computer with a keyboard, display, and network connection is sufficient. The project does not require specialized hardware.

4.2 Software requirements

A supported Python installation, a terminal, and an IDE such as Visual Studio Code are sufficient. A virtual environment is recommended.

4.3 Functional requirements

FR-01: accept a topic. FR-02: strip whitespace. FR-03: reject empty input. FR-04: build encoded parameters.

FR-05: send the API request. FR-06: parse JSON. FR-07: extract results. FR-08: select up to ten articles.

FR-09: display title and description.

4.4 Non-functional requirements

The system should be usable, maintainable, portable, reasonably responsive, reliable for expected API responses, and secure with respect to credentials.

4.5 User requirements

The user needs only to enter a meaningful news topic. No complex configuration is required beyond securely providing the API credential.

4.6 Environmental requirements

Because the project retrieves live information, internet access is necessary during normal operation. Offline parsing tests can be performed using saved JSON fixtures.

ID	Functional Requirement
FR-01	Accept news topic
FR-02	Validate input
FR-03	Construct API request
FR-04	Retrieve API response
FR-05	Parse JSON
FR-06	Extract results
FR-07	Display maximum 10 articles

CHAPTER 5 — SYSTEM DESIGN

5.1 Architecture

The architecture is: USER → INPUT → VALIDATION → PARAMETER ENCODING → HTTP CLIENT → NEWSDATA.IO → JSON RESPONSE → PARSER → ARTICLE DISPLAY.

5.2 Workflow

The user starts the program, enters a topic, and the application validates it. It then builds the query, sends the request, receives JSON, extracts the results, selects the first ten records, and displays them.

5.3 Data flow

Topic → encoded query parameters → HTTP request → API response → decoded JSON → Python dictionary/list → article fields → terminal output.

5.4 Module design

The current file is compact, but its responsibilities can be logically separated into input handling, API client, JSON parser, result formatter, and error-handling layers.

5.5 Input design

Input is collected using input() and normalized using strip(). An empty string is considered invalid.

5.6 Output design

The output begins with the selected news type and then prints each article using a number, title, and description.

5.7 Architecture limitation

The current architecture does not include a database, web interface, speech recognition, Text-to-Speech, or AI generation layer.

System architecture diagram:

USER → INPUT → VALIDATION → PARAMETER ENCODING → HTTP CLIENT → NEWSDATA.IO → JSON → PARSER → FIRST 10 ARTICLES → CONSOLE

Data-flow diagram:

Topic → Query Parameters → API Request → JSON Response → Python Object → results[] → title/description → Output

CHAPTER 6 — REST API AND NEWSDATA.IO INTEGRATION

6.1 API concept

An API defines a structured way for software components to communicate. In this project, the Python program acts as the client and the news provider acts as the server.

6.2 Request construction

The program creates a parameter dictionary containing the API key, user query, and language. These values are converted to a URL-encoded query string.

6.3 URL encoding

URL encoding is important because user queries can contain spaces and special characters. urlencode() converts such values into a valid query representation.

6.4 HTTP request

urllib.request.urlopen() opens the constructed URL and returns a response object. The response body is read as bytes and decoded into text.

6.5 JSON response

The API returns structured JSON. json.loads() converts the JSON text into Python data structures.

6.6 Provider dependency

The exact endpoint, parameters, quotas, and response schema are controlled by NewsData.io and may change. The application should therefore be maintained against the provider's current documentation.

6.7 Request-response cycle

Client constructs request → server authenticates and processes request → server returns response → client decodes response → client extracts required information.

6.8 API integration benefits

The integration allows the project to retrieve current external information without maintaining a local news database.

Representative implementation fragment:

```
params = urllib.parse.urlencode({"apikey": api_key, "q": query,
"language": "en"}) url = f"https://newsdata.io/api/1/news?{params}"
with urllib.request.urlopen(url) as response:
data = json.loads(response.read().decode("utf-8"))
```

CHAPTER 7 — PYTHON IMPLEMENTATION

7.1 Imports

The implementation imports json for parsing, urllib.parse for query encoding, and urllib.request for the HTTP request.

7.2 User input

The input() function prompts the user for the type of news. strip() removes unnecessary spaces from the beginning and end.

7.3 Validation

The condition if not query detects an empty query. A clear message is displayed and SystemExit(1) terminates execution.

7.4 API key

The API key authenticates the request. For production and GitHub use, it should be loaded from an environment variable rather than hard-coded.

7.5 Parameter encoding

The parameters dictionary is passed to urllib.parse.urlencode(). This converts the values into a URL-compatible query string.

7.6 URL construction

The encoded query is appended to the API endpoint using an f-string.

7.7 Request

urllib.request.urlopen(url) sends the HTTP request and provides access to the response body.

7.8 Response decoding

response.read() obtains bytes. decode('utf-8') converts those bytes to readable text.

7.9 JSON parsing

json.loads() converts the JSON text into Python objects such as dictionaries and lists.

7.10 Result selection

data.get('results', []) safely obtains the results list. [:10] restricts the output to the first ten records.

7.11 Article extraction

article.get('title', 'No title') and article.get('description', 'No description') safely access fields.

7.12 Presentation

The program prints a numbered result with the title and description, making the console output easy to read.

Key implementation fragment:

```
query = input("Enter the types of news: ").strip()
if not query:
print("Please enter a valid news
type.") raise SystemExit(1)
articles = data.get("results", [])[:10]
```

CHAPTER 8 — JSON DATA PROCESSING

8.1 JSON fundamentals

JSON represents data using objects, arrays, strings, numbers, booleans, and null values. API services frequently use JSON because it is compact and widely supported.

8.2 Python mapping

A JSON object is generally converted to a Python dictionary and a JSON array to a Python list.

8.3 Parsing

After the response is decoded, json.loads() parses the complete response.

8.4 Results collection

The application accesses the results key, which contains the article records returned by the service.

8.5 Safe access

The get() method allows fallback values if a field is absent. This prevents a missing title or description from immediately causing a KeyError.

8.6 First ten records

Slicing the results list using [:10] returns at most ten records. If fewer than ten are available, all available records are returned. **8.7 Data quality**

External API data can be incomplete. Defensive field access is therefore important for reliable presentation.

8.8 Processing summary

JSON text → Python dictionary → results list → individual article dictionary → title and description.

JSON concept	Python representation
Object	dict
Array	list
String	str
Number	int/float
Boolean	bool
Null	None

CHAPTER 9 — INPUT VALIDATION AND ERROR HANDLING

9.1 Importance

Validation prevents the application from sending an unnecessary or meaningless request.

9.2 Empty input

strip() ensures that input containing only spaces becomes an empty string and is rejected.

9.3 Current validation

The current code prints 'Please enter a valid news type.' and exits with status 1 when the query is empty.

9.4 Network errors

A production version should catch network-related exceptions such as URL or connection failures and display a useful message.

9.5 HTTP errors

Invalid credentials, unavailable endpoints, and quota issues may produce HTTP errors. These should be handled separately where possible.

9.6 JSON errors

If a server returns malformed JSON or an unexpected body, JSON decoding can fail. The program should handle such failures in a robust version.

9.7 Timeouts

A timeout prevents the application from waiting indefinitely for a slow network response.

9.8 Future validation

Future versions can add query length limits, allowed-character rules, retry strategies, and clearer API-status messages.

CHAPTER 10 — SECURITY AND API-KEY MANAGEMENT

10.1 Security importance

The API key is a credential that authorizes access to the external service. Exposing it publicly can allow unauthorized usage and may exhaust the account's quota.

10.2 Current risk

A hard-coded key in source code is unsafe when the repository is public. It should be replaced before GitHub publication.

10.3 Environment variable

A recommended pattern is **api_key = os.getenv('NEWSDATA_API_KEY')**. The key can then be configured locally without placing it in the source.

10.4 .gitignore

Local secret files such as .env should be excluded from version control. The repository should contain a safe example configuration rather than the actual credential.

10.5 Rotation

If a key has already been committed to a public repository, it should be considered compromised and rotated through the provider.

10.6 Secure error messages

Error messages should not print the full API URL when it contains a secret key.

10.7 Repository security

Only source code, documentation, tests, and safe configuration examples should be committed.

Recommended secure configuration:

```
import os api_key =
os.getenv("NEWSDATA_A
PI_KEY")
```

CHAPTER 11 — TESTING AND TEST CASES

11.1 Testing objective

Testing verifies that the program behaves correctly for normal, invalid, and unexpected conditions.

11.2 Functional testing

Functional testing checks whether input, validation, API retrieval, JSON parsing, selection, and output work as intended.

11.3 Integration testing

Integration testing verifies communication between the Python application and NewsData.io.

11.4 Negative testing

Negative tests include empty input, invalid credentials, unavailable network, malformed data, and unexpected fields.

11.5 Test cases

A professional test suite should record test ID, input, expected result, actual result, and status.

11.6 Recommended test table

TC01 technology → news displayed. TC02 sports → news displayed. TC03 business → news displayed. TC04 empty input → validation message. TC05 query with spaces → encoded request. TC06 ten or more articles → exactly ten displayed. TC07 fewer than ten → all available displayed. TC08 missing title → fallback text. TC09 missing description → fallback text. TC10 API/network failure → controlled error in enhanced version.

Test	Input	Expected Result
TC01	technology	News displayed
TC02	sports	News displayed
TC03	<empty>	Validation message
TC04	AI news	Results displayed
TC05	10+ results	Maximum 10
TC06	<10 results	All available

CHAPTER 12 — RESULTS AND DISCUSSION

12.1 Execution

When the application is executed, it prompts the user to enter a news topic.

12.2 Sample input

Example: **technology**. The input is stripped and validated before the API request is created.

12.3 Sample output

The terminal displays the selected news type followed by numbered articles. Each item contains a title and description.

12.4 Result analysis

The first ten records are selected using Python list slicing. The use of get() makes the output more tolerant of missing fields.

12.5 User experience

The command-line interface is simple and requires minimal interaction. The user does not need to manually construct API URLs.

12.6 Discussion

The project achieves its primary objective of demonstrating automated topic-based news retrieval. Its simplicity is an advantage for learning, while also defining clear areas for future enhancement.

CHAPTER 13 — GITHUB DOCUMENTATION

13.1 Repository purpose

GitHub provides a professional platform for source-code hosting, version control, documentation, and project presentation.

13.2 Recommended structure

AI-Power-News-Assistant/ with news_assistant.py, README.md, requirements.txt where applicable, .gitignore, LICENSE, docs/, and tests/.

13.3 README contents

The README should include project title, description, features, technologies, prerequisites, installation, configuration, usage, sample output, limitations, future scope, and security notes.

13.4 Installation

Document Python installation, virtual-environment creation, dependency setup if required, environment-variable configuration, and execution command.

13.5 Usage

Explain that the user runs the Python file, enters a topic, and reviews up to ten returned articles.

13.6 Security

Never place the actual API key in README.md, source code, screenshots, or commit history.

13.7 Version control

Use meaningful commits such as 'Add NewsData API integration', 'Add input validation', and 'Improve README documentation'.

13.8 Professional presentation

A good GitHub repository should clearly communicate what the project does and, equally importantly, what it does not do.

Recommended GitHub structure:

AI-Power-News-Assistant/

- news_assistant.py
- README.md
- requirements.txt
- .gitignore
- LICENSE
- docs/
- project-report.pdf
- tests/

CHAPTER 14 — LIMITATIONS

14.1 Internet dependency

The application requires an internet connection to retrieve current news.

14.2 API dependency

The application depends on NewsData.io availability, response structure, authentication rules, and service limits.

14.3 Credential dependency

A valid API key is required for successful authenticated requests.

14.4 Interface limitation

The current interface is command-line based and does not provide a graphical or web

interface. **14.5 Data limitation**

The program displays retrieved title and description fields; it does not independently verify article accuracy.

14.6 AI limitation

Despite the project name, the current implementation does not perform generative AI summarization or reasoning over the retrieved news.

14.7 Voice limitation

Voice-based search, speech recognition, Text-to-Speech, and audio news are not included.

14.8 Scalability limitation

The current single-file design is suitable for a small project but should be modularized for a larger production system.

CHAPTER 15 — FUTURE SCOPE, CONCLUSION AND REFERENCES

15.1 Future scope

Future development may include AI-based summarization, category filtering, duplicate detection, sentiment analysis, multilingual support, personalized topics, search history, scheduled retrieval, database storage, Flask/Streamlit interfaces, automated testing, and CI/CD.

15.2 Voice as future scope

Voice search and Text-to-Speech can be added later as separate modules. They are not part of the current implementation and should not be presented as existing features.

15.3 Professional conclusion

AI Power News Assistant demonstrates a complete foundational REST API integration workflow using Python. It accepts user input, validates the topic, constructs an encoded request, retrieves JSON from NewsData.io, processes the response, and displays up to ten articles.

15.4 Project value

The project provides practical experience in Python, HTTP communication, REST APIs, JSON, URL encoding, input validation, defensive data access, security, testing, and GitHub documentation.

15.5 References

Recommended references include Python official documentation; NewsData.io API documentation; documentation for json, urllib.parse, and urllib.request; Git documentation; GitHub documentation; and Visual Studio Code documentation.

15.6 Final note

The project should be presented accurately: it is a text-based topic-driven news retrieval application. Voice search, Text-to-Speech, audio news, and AI-generated summaries are future possibilities rather than current features.

APPENDIX A — COMPLETE PROGRAM LISTING

The following listing reflects the core implementation supplied for the project. It intentionally contains no voice functionality.

Python source

```
import json
import urllib.parse
import urllib.request
query = input("Enter the types of news: ").strip()
if not query:
    print("Please enter a valid news type.")
    raise SystemExit(1)
api_key = "pub_729f61bdca274b4397b979599b16cccc"
params = urllib.parse.urlencode({"apikey": api_key, "q": query, "language": "en"})
url = f"https://newsdata.io/api/1/news?{params}"
with urllib.request.urlopen(url) as response:
    data = json.loads(response.read().decode("utf-8"))
print(f"News type: {query}")
articles = data.get("results", [])[:10]  # Get first 10 articles
for i, article in enumerate(articles, start=1):
```

```
    title = article.get("title", "No title")
    description = article.get("description", "No description")
    print(f"\n{i}. Title: {title}")
    print(f"   Description: {description}")
```

APPENDIX B — LINE-BY-LINE CODE NOTES

Imports

json supports JSON parsing. urllib.parse provides URL encoding. urllib.request provides HTTP requests.

Input

input() collects the user's news type. strip() removes surrounding whitespace.

Validation

The empty-string check prevents a meaningless API request.

Authentication

The API key authenticates access and should be stored securely.

Parameters

The query, API key, and language are placed into a dictionary.

Encoding

urlencode() converts the dictionary into a URL-compatible query string.

Request

urlopen() sends the HTTP request.

Response

read() obtains the response body and decode() converts it to text.

Parsing

json.loads() converts JSON into Python structures.

Selection

get('results', []) safely obtains the article list and [:10] limits the output.

Display

enumerate() numbers the articles and get() extracts title and description.

APPENDIX C — API REQUEST-RESPONSE CYCLE

Client side

The Python program is the client. It constructs a request containing authentication and search parameters.

Transport

The request is sent over HTTP/HTTPS to the API endpoint.

Server side

The provider authenticates the request, processes the query, and prepares a response.

Response

The provider returns JSON containing status information and article results.

Client processing

Python reads, decodes, parses, and extracts the required fields.

Output

The terminal presents the selected results to the user.

APPENDIX D — DETAILED FUNCTIONAL REQUIREMENTS

FR-01: The system shall prompt the user for a news topic.

FR-02: The system shall remove leading and trailing whitespace.

FR-03: The system shall reject an empty topic.

FR-04: The system shall encode API query parameters.

FR-05: The system shall request news from the configured API endpoint.

FR-06: The system shall parse the returned JSON.

FR-07: The system shall access the results collection safely.

FR-08: The system shall display no more than ten articles.

FR-09: The system shall display title and description for each selected article.

APPENDIX E — NON-FUNCTIONAL REQUIREMENTS

Usability

Output should be readable and understandable to a beginner.

Reliability

Expected successful API responses should be processed consistently.

Security

API credentials should be protected.

Maintainability

The code should remain simple and logically organized.

Portability

The program should run on common Python-supported operating systems.

Performance

The application should avoid unnecessary requests and processing.

Scalability

The design should permit future separation into modules.

APPENDIX F — EXISTING VS PROPOSED SYSTEM

Manual browsing

User repeatedly searches websites and reads results manually.

Proposed application

User enters one topic and receives structured results.

Data source

Manual browsing may use many sources; application uses configured API.

Processing

Manual inspection versus automated JSON extraction.

Output

Web pages versus numbered console results.

Extensibility

Manual workflow versus programmable future modules.

APPENDIX G — SYSTEM WORKFLOW DESCRIPTION

Step 1: Start the Python program.

Step 2: Display the input prompt.

Step 3: Receive the topic.

Step 4: Strip whitespace.

Step 5: Validate that the topic is not empty.

Step 6: Construct request parameters.

Step 7: Encode parameters.

Step 8: Build the endpoint URL.

Step 9: Send the request.

Step 10: Read and decode the response.

Step 11: Parse JSON.

Step 12: Extract results.

Step 13: Select first ten records.

Step 14: Print title and description.

Step 15: End.

APPENDIX H — INPUT AND OUTPUT DESIGN

Input example

technology

Input example

artificial intelligence

Input example

sports

Invalid input

empty string or whitespace only

Output header

News type: technology

Output item

1. Title:

Output detail

Description:

Output count

Maximum ten articles.

APPENDIX I — JSON FIELD HANDLING

results

Primary collection from which articles are

selected. **title**

Headline displayed to the user.

description

Article summary/description supplied by the API.

Fallback title

No title.

Fallback description

No description.

Safe access

dict.get() avoids an immediate exception when a field is absent.

APPENDIX J — ERROR SCENARIOS

Empty input

Prevent request and display validation message.

Invalid API key

Provider may reject the request; enhanced version should display a controlled error.

Rate limit

Provider may restrict requests; application should avoid excessive retries.

Network unavailable

Request can fail; enhanced version should catch the relevant exception.

Malformed JSON

Parsing can fail; enhanced version should handle decoding errors.

Unexpected schema

Missing results or fields should be handled safely.

APPENDIX K — SECURITY CHECKLIST

Do not publish the real API key.

Use an environment variable for credentials.

Add secret files to .gitignore.

Do not print credentials in error messages.

Rotate exposed credentials.

Review Git history before making a repository public.

Document safe configuration in README.md.

Follow the API provider’s authentication and usage rules.

- Use environment variables
- Keep secrets outside source control
- Rotate compromised keys
- Audit public repositories

APPENDIX L — TEST EXECUTION PLAN

Phase 1

Test input and validation without contacting the API.

Phase 2

Test successful API retrieval using a valid credential.

Phase 3

Test result extraction and output formatting.

Phase 4

Test missing fields using controlled JSON fixtures.

Phase 5

Test network and HTTP error handling in an enhanced implementation.

Phase 6

Review security and repository configuration.

APPENDIX M — TEST REPORT TEMPLATE

Test ID | Date | Input | Expected Result | Actual Result | Status

TC01 | — | technology | News displayed | — | —

TC02 | — | sports | News displayed |

— | — TC03 | — | empty | Validation

message | — | —

TC04 | — | artificial intelligence | Results displayed | — | —

TC05 | — | query returning 10+ | Ten displayed |

— | — TC06 | — | query returning <10 | Available

displayed | — | —

APPENDIX N — GITHUB README CONTENT

Project title

AI Power News Assistant

Short description

A Python-based topic-driven news retrieval application using NewsData.io REST API.

Features

Topic search, REST API integration, JSON processing, validation, first-ten article display.

Technologies

Python, json, urllib, NewsData.io.

Installation

Install Python, create a virtual environment, configure the API key securely, and run the script.

Usage

Enter a news topic when prompted.

Security

Never commit the real API key.

Limitations

Text-based command-line retrieval only.

Future scope

AI summarization, web UI, analytics, and optional voice features.

APPENDIX O — PROFESSIONAL PROJECT DESCRIPTION

AI Power News Assistant is a lightweight Python application designed to retrieve topic-specific news using the NewsData.io REST API. It accepts a topic from the user, validates the input, creates URL-encoded request parameters, retrieves a JSON response, and displays the first ten article titles and descriptions.

The project demonstrates practical knowledge of Python standard libraries, REST API communication, JSON data processing, input validation, secure credential management, testing, and GitHub documentation.

The current version is intentionally text-based. Voice search, Text-to-Speech, audio news, and AI-generated summarization are not implemented.

APPENDIX P — PROJECT BENEFITS

Educational benefit: demonstrates practical API integration.

Programming benefit: strengthens Python and structured-data skills.

API benefit: teaches request construction and response handling.

Security benefit: introduces API-key protection.

Documentation benefit: creates a professional GitHub-ready project.

Extension benefit: provides a foundation for future web, analytics, and AI modules.

APPENDIX Q — RISK MANAGEMENT

Risk

API outage

Risk

Key exposure

Risk

Rate limit

Risk

Schema change

Risk

Network failure

Risk	Impact	Mitigation
API outage	No live news	Controlled error
Key exposure	Unauthorized use	Environment variable
Rate limit	Rejected request	Respect quota
Schema change	Parsing failure	Validate response
Network failure	No response	Timeout/exception

APPENDIX R — FINAL PROJECT CHECKLIST

Source code runs successfully.

User input is validated.

API parameters are encoded.

JSON response is parsed.

First ten results are selected.

Title and description are displayed.

API key is removed from public source.

.gitignore is configured.

README.md is complete.

Testing is documented.

Limitations are stated accurately.

Future scope is separated from current features.

Report contains architecture, requirements, implementation, testing, security, and conclusion.